Conjugate Gradient

Suppose we want to solve the system of linear equations

$$\mathbf{A}\mathbf{x} = \mathbf{b}$$

for the vector $\mathbf{x}$, where the known $n \times n$ matrix $\mathbf{A}$ is symmetric (i.e., $\mathbf{A}^T = \mathbf{A}$), positive-definite (i.e. $\mathbf{x}^T\mathbf{A}\mathbf{x} > 0$ for all non-zero vectors $\mathbf{x} \in \mathbb{R}^n$), and real, and $\mathbf{b}$ is known as well. We denote the unique solution of this system by $\mathbf{x}^*$.

Solution of the above system of equations is the same as $\operatorname{argmin}_{\mathbf{x}\in\mathbb{R}^n} \frac{1}{2}\mathbf{x}^T\mathbf{A}\mathbf{x} - \mathbf{b}^T\mathbf{x}$.

Solving such a system also comes naturally in Newton–Raphson optimization when the Hessian is positive definite. (Verify!)

Definition 1 (**A**-conjugate directions)**.** *Let* $\mathbf{A}$ *$(n \times n)$ be a symmetric matrix. The vectors* $\{\mathbf{d}_1, \ldots, \mathbf{d}_n\}$ *(all* $\in \mathbb{R}^n$*) are called conjugate (or orthogonal) with respect to* $\mathbf{A}$ *if* $\mathbf{d}_i^T\mathbf{A}\mathbf{d}_j = 0$ *for all* $i \neq j$*.*

Lemma 1 (Linear independence)**.** *Let* $\mathbf{A}$ *$(n \times n)$ be positive definite. If the nonzero vectors* $\{\mathbf{d}_1, \ldots, \mathbf{d}_k\}$ *(all* $\in \mathbb{R}^n$*) are conjugate (orthogonal) with respect to* $\mathbf{A}$*, then they are linearly independent.*

Proof. Suppose the vectors are linearly dependent. After relabeling if needed, write $\mathbf{d}_k = \sum_{j=1}^{k-1} \alpha_j \mathbf{d}_j$.

By definition of conjugacy, we have $\mathbf{d}_k^T\mathbf{A}\mathbf{d}_j = 0$ for all $j \neq k$. Since, $\mathbf{A}$ is pd, we have $\mathbf{d}_k^T\mathbf{A}\mathbf{d}_k > 0$. On the other hand, $\mathbf{d}_k^T\mathbf{A}\mathbf{d}_k = \sum_{j=1}^{k-1} \alpha_j \mathbf{d}_k^T\mathbf{A}\mathbf{d}_j = 0$ (Contradiction!) □

If $\mathbf{x}^*$ is a solution of the above system of equation, we can represent it as a linear combination of $\{\mathbf{d}_1, \ldots, \mathbf{d}_n\}$, which are conjugate vectors with respect to $\mathbf{A}$.

Let $\mathbf{x}^* = \sum_{j=1}^{n} \alpha_j \mathbf{d}_j$. It is easy to check that $\mathbf{d}_j^T\mathbf{A}\mathbf{x}^* = \alpha_j \mathbf{d}_j^T\mathbf{A}\mathbf{d}_j$. Thus $\alpha_j = \frac{\mathbf{d}_j^T\mathbf{A}\mathbf{x}^*}{\mathbf{d}_j^T\mathbf{A}\mathbf{d}_j}$. But by assumption, $\mathbf{A}\mathbf{x}^* = \mathbf{b}$. Thus $\alpha_j = \frac{\mathbf{d}_j^T\mathbf{b}}{\mathbf{d}_j^T\mathbf{A}\mathbf{d}_j}$. Hence, to know α_j, we do not need to know $\mathbf{x}^*$.

Hence $\mathbf{x}^* = \sum_{j=1}^{n} \frac{\mathbf{d}_j^T\mathbf{b}}{\mathbf{d}_j^T\mathbf{A}\mathbf{d}_j}\mathbf{d}_j$, no matrix inversion or nothing.

In practice, we do not know the conjugate vectors. We identify them iteratively using a Gram–Schmidt type construction. Each iterate minimizes the quadratic over the affine space generated by the directions used so far. We can stop early when the residual is

sufficiently small. There is no general ordering $|\alpha_1| \geq |\alpha_2| \geq \cdots$; the coefficients also depend on the scaling of the directions.

Algorithm 1: Conjugate gradient

(i) Start setting $\mathbf{r}_1 = \mathbf{b} - \mathbf{A}\mathbf{x}_1$ with some starting value $\mathbf{x}_1$.
(ii) If $\mathbf{r}_1$ is small enough, return $\mathbf{x} = \mathbf{x}_1$ as solution.
(iii) Otherwise, set $\mathbf{d}_1 = \mathbf{r}_1$ and $k = 1$.
(iv) Start loop

- $\alpha_k = \frac{\mathbf{r}_k^T \mathbf{r}_k}{\mathbf{d}_k^T \mathbf{A} \mathbf{d}_k}$
- $\mathbf{x}_{k+1} = \mathbf{x}_k + \alpha_k \mathbf{d}_k$
- $\mathbf{r}_{k+1} = \mathbf{r}_k - \alpha_k \mathbf{A} \mathbf{d}_k$
- If $\mathbf{r}_{k+1}$ is small enough, return $\mathbf{x} = \mathbf{x}_{k+1}$ as solution.
- Otherwise, $\beta_k = \frac{\mathbf{r}_{k+1}^T \mathbf{r}_{k+1}}{\mathbf{r}_k^T \mathbf{r}_k}$, $\mathbf{d}_{k+1} = \mathbf{r}_{k+1} + \beta_k \mathbf{d}_k$, move $k = k + 1$ and Repeat

In the above algorithm, how the term 'gradient' fits in? Say $f(\mathbf{x}) = \frac{1}{2}\mathbf{x}^T \mathbf{A} \mathbf{x} - \mathbf{b}^T \mathbf{x}$ and we are trying to solve $\min_{\mathbf{x}} f(\mathbf{x})$. Following the steps in gradient descent, the updates should look like $\mathbf{x}_{k+1} = \mathbf{x}_k - t_k \bigtriangledown f(\mathbf{x}_k)$. For our function, we have $\bigtriangledown f(\mathbf{x}) = \mathbf{A}\mathbf{x} - \mathbf{b}$. Thus, we need to move towards $\mathbf{b} - \mathbf{A}\mathbf{x}$. The other vectors in the basis will be made conjugate to this gradient vector. Thus, the term 'conjugate gradient' comes in.

To see above, note that $\beta_k = \frac{\mathbf{r}_{k+1}^T \mathbf{r}_{k+1}}{\mathbf{r}_k^T \mathbf{r}_k} = -\frac{\mathbf{r}_{k+1}^T \mathbf{A} \mathbf{d}_k}{\mathbf{d}_k^T \mathbf{A} \mathbf{d}_k}$. Then, $\mathbf{d}_{k+1}^T \mathbf{A} \mathbf{d}_k = 0$. In exact arithmetic, linear CG reaches the solution in at most n iterations; rounding errors can require additional iterations.

If the condition number of $\mathbf{A}$ is large, the convergence is slow as it leads to slower improvement. Hence, to ensure faster convergence in this case, **Preconditioning conjugate gradient** is used. [Condition number of a matrix $\mathbf{A}$ is $\kappa(\mathbf{A}) = \frac{\sigma_{\max}(\mathbf{A})}{\sigma_{\min}(\mathbf{A})}$, where $\sigma_{\max}(\mathbf{A})$ and $\sigma_{\min}(\mathbf{A})$ are maximal and minimal singular values of $\mathbf{A}$ respectively. In our case, the singular values and eigenvalues are the same as $\mathbf{A}$ is symmetric]

In the example codes, compare the objective values and residual norms across iterations. The objective decreases, but the residual norm need not decrease at every iteration. Also, see how the condition number of $\mathbf{A}$ affects convergence.

Algorithm 2: Pre-conditioned conjugate gradient

$$\begin{aligned}
\mathbf{r}_0 &= \mathbf{b} - \mathbf{A}\mathbf{x}_0 \\
\mathbf{z}_0 &= \mathbf{M}^{-1}\mathbf{r}_0 \\
\mathbf{p}_0 &= \mathbf{z}_0 \\
k &= 0
\end{aligned}$$

If $\|\mathbf{r}_0\|$ is sufficiently small, return $\mathbf{x}_0$. Otherwise, repeat

$$\begin{aligned}
\alpha_k &= \frac{\mathbf{r}_k^\mathsf{T}\mathbf{z}_k}{\mathbf{p}_k^\mathsf{T}\mathbf{A}\mathbf{p}_k} \\
\mathbf{x}_{k+1} &= \mathbf{x}_k + \alpha_k\mathbf{p}_k \\
\mathbf{r}_{k+1} &= \mathbf{r}_k - \alpha_k\mathbf{A}\mathbf{p}_k
\end{aligned}$$

if $\|\mathbf{r}_{k+1}\|$ is sufficiently small, then exit loop end if

$$\begin{aligned}
\mathbf{z}_{k+1} &= \mathbf{M}^{-1}\mathbf{r}_{k+1} \\
\beta_k &= \frac{\mathbf{r}_{k+1}^T\mathbf{z}_{k+1}}{\mathbf{r}_k^T\mathbf{z}_k} \\
\mathbf{p}_{k+1} &= \mathbf{z}_{k+1} + \beta_k\mathbf{p}_k \\
k &= k+1,
\end{aligned}$$

end repeat;
The result is $\mathbf{x}_{k+1}$

In the above algorithm $\mathbf{M}$ is fixed, symmetric, and positive definite. We often set $\mathbf{M} = \mathbf{L}\mathbf{L}^T$, where $\mathbf{L}$ is an incomplete Cholesky factor of $\mathbf{A}$, or use the diagonal of $\mathbf{A}$ (Jacobi preconditioner). Computing $\mathbf{z}_k = \mathbf{M}^{-1}\mathbf{r}_k$ means solving $\mathbf{M}\mathbf{z}_k = \mathbf{r}_k$; we need not form an inverse. For clarity, we are applying basic CG to $\mathbf{L}^{-1}\mathbf{A}\mathbf{L}^{-T}\mathbf{y} = \mathbf{L}^{-1}\mathbf{b}$ and recovering $\mathbf{x} = \mathbf{L}^{-T}\mathbf{y}$. The preconditioner should make the transformed system better conditioned while remaining inexpensive to solve.

Nonlinear conjugate gradient

Suppose we now want to minimize a differentiable function $f(\mathbf{x})$ that need not be quadratic. Write $\mathbf{g}_k = \nabla f(\mathbf{x}_k)$. Gradient descent uses

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \alpha_k \mathbf{g}_k.$$

At every iteration, the direction is determined entirely by the current gradient. Can we improve this by retaining some information from the previous direction?

Why keep the previous direction?

Consider the quadratic function

$$f(\mathbf{x}) = \frac{1}{2}(x_1^2 + 100x_2^2), \qquad \nabla f(\mathbf{x}) = \begin{pmatrix} x_1 \\ 100x_2 \end{pmatrix}.$$

Its contours form a narrow valley. With a fixed step size $\alpha = 0.018$, gradient descent gives

$$x_{1,k+1} = 0.982x_{1,k}, \qquad x_{2,k+1} = -0.8x_{2,k}.$$

Thus, the second coordinate oscillates across the valley, while the first coordinate moves slowly toward zero. A step size small enough for the steep direction can give slow progress along the shallow direction.

The successive steps contain useful information: their components along the valley tend to agree, while their components across the valley can alternate in sign. Momentum retains part of the preceding step:

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \eta_k \mathbf{g}_k + \mu_k(\mathbf{x}_k - \mathbf{x}_{k-1}).$$

For positive μ_k, repeated movement in the same direction is reinforced, while opposing components can partially cancel. This can reduce zigzagging and improve progress along the valley. The choices of η_k and μ_k matter; excessive momentum can cause overshooting.

CG already has a momentum term.

In linear CG, $\mathbf{r}_k = -\mathbf{g}_k$. Hence its direction update can be written as

$$\mathbf{d}_k = -\mathbf{g}_k + \beta_{k-1}\mathbf{d}_{k-1}, \qquad k \geq 2.$$

The new direction combines the current negative gradient with the previous search direction. Since

$$\mathbf{x}_k - \mathbf{x}_{k-1} = \alpha_{k-1}\mathbf{d}_{k-1},$$

the update $\mathbf{x}_{k+1} = \mathbf{x}_k + \alpha_k \mathbf{d}_k$ becomes

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \alpha_k \mathbf{g}_k + \frac{\alpha_k \beta_{k-1}}{\alpha_{k-1}} (\mathbf{x}_k - \mathbf{x}_{k-1}). \tag{1}$$

This has the momentum form, with coefficient

$$\mu_k = \frac{\alpha_k \beta_{k-1}}{\alpha_{k-1}}.$$

Thus, β_{k-1} weights the previous search direction, whereas μ_k weights the previous displacement. When the step sizes agree, $\mu_k = \beta_{k-1}$; otherwise, their ratio also enters the displacement coefficient. The CG coefficient is computed from the optimization problem at each iteration.

Why does CG choose this particular amount of the previous direction? For the quadratic function, requiring $\mathbf{d}_{k+1}^T \mathbf{A} \mathbf{d}_k = 0$ gives

$$0 = (-\mathbf{g}_{k+1} + \beta_k \mathbf{d}_k)^T \mathbf{A} \mathbf{d}_k, \qquad \beta_k = \frac{\mathbf{g}_{k+1}^T \mathbf{A} \mathbf{d}_k}{\mathbf{d}_k^T \mathbf{A} \mathbf{d}_k}.$$

With the exact line searches of linear CG, this equals

$$\beta_k = \frac{\mathbf{g}_{k+1}^T \mathbf{g}_{k+1}}{\mathbf{g}_k^T \mathbf{g}_k}.$$

Thus, the retained direction is chosen using conjugacy, rather than an arbitrary fixed weight.

To see why conjugacy helps, an exact line search along $\mathbf{d}_k$ gives $\mathbf{g}_{k+1}^T \mathbf{d}_k = 0$: the function has already been minimized along that direction. For a quadratic, after the next step,

$$\mathbf{g}_{k+2} = \mathbf{g}_{k+1} + \alpha_{k+1} \mathbf{A} \mathbf{d}_{k+1}.$$

Consequently,

$$\mathbf{d}_k^T \mathbf{g}_{k+2} = \mathbf{d}_k^T \mathbf{g}_{k+1} + \alpha_{k+1} \mathbf{d}_k^T \mathbf{A} \mathbf{d}_{k+1} = 0.$$

The new step preserves the minimization already achieved along $\mathbf{d}_k$. This explains how retaining the previous direction can help avoid repeatedly correcting earlier movement.

Extending the update to a nonlinear function.

For a nonlinear f, the Hessian generally changes with $\mathbf{x}$, so there is no single fixed matrix $\mathbf{A}$ with respect to which all directions remain conjugate. We nevertheless use the same recurrence:

$$\begin{aligned}
\mathbf{d}_1 &= -\mathbf{g}_1, \\
\mathbf{x}_{k+1} &= \mathbf{x}_k + \alpha_k \mathbf{d}_k, \\
\mathbf{g}_{k+1} &= \nabla f(\mathbf{x}_{k+1}), \\
\mathbf{d}_{k+1} &= -\mathbf{g}_{k+1} + \beta_k \mathbf{d}_k.
\end{aligned}$$

Equation (1) still holds. Nonlinear CG therefore retains the momentum structure, but adjusts the previous direction using newly computed gradients. We must evaluate the gradient at the new point; the linear residual update cannot generally be used.

Two common choices for β_k are:

1. Fletcher–Reeves (FR):
$$\beta_k^{\mathrm{FR}} = \frac{\mathbf{g}_{k+1}^T \mathbf{g}_{k+1}}{\mathbf{g}_k^T \mathbf{g}_k}.$$
This carries the linear CG formula directly to a nonlinear objective.

2. Polak–Ribière (PR):
$$\beta_k^{\mathrm{PR}} = \frac{\mathbf{g}_{k+1}^T (\mathbf{g}_{k+1} - \mathbf{g}_k)}{\mathbf{g}_k^T \mathbf{g}_k}.$$
This also uses the change in the gradient. If $\mathbf{g}_{k+1}$ is nearly equal to $\mathbf{g}_k$, the numerator is small, so little of the previous direction is retained. A common modification is
$$\beta_k^{\mathrm{PR}+} = \max\{0, \beta_k^{\mathrm{PR}}\}.$$
A negative PR coefficient therefore triggers a return to the negative gradient direction.

For a positive definite quadratic with exact line searches, successive gradients are orthogonal, and FR and PR give the same coefficient. For a general nonlinear function, they need not agree. Neither FR nor PR+ restricts β_k to be less than one.

Step size and restarting.

The closed-form step size from linear CG is generally unavailable. Instead, consider the one-dimensional function
$$\phi_k(\alpha) = f(\mathbf{x}_k + \alpha \mathbf{d}_k), \qquad \phi_k'(\alpha) = \nabla f(\mathbf{x}_k + \alpha \mathbf{d}_k)^T \mathbf{d}_k.$$
An exact line search would find a minimizer of ϕ_k. In practice, an approximate line search is sufficient. A common choice is to require the strong Wolfe conditions:
$$\begin{aligned} f(\mathbf{x}_k + \alpha_k \mathbf{d}_k) &\le f(\mathbf{x}_k) + c_1 \alpha_k \mathbf{g}_k^T \mathbf{d}_k, \\ \left| \nabla f(\mathbf{x}_k + \alpha_k \mathbf{d}_k)^T \mathbf{d}_k \right| &\le c_2 \left| \mathbf{g}_k^T \mathbf{d}_k \right|, \end{aligned}$$
where $0 < c_1 < c_2 < 1$ and $\mathbf{g}_k^T \mathbf{d}_k < 0$. For example, use $c_1 = 10^{-4}$ and $c_2 = 0.1$. The first condition requires sufficient decrease. The second requires the directional slope to be substantially reduced, approximating the zero slope obtained by exact line search. A Wolfe

line search brackets and refines a suitable step; simply shrinking a step until the objective decreases does not enforce the second condition.

The previous direction can become unhelpful as the local curvature changes. Indeed,

$$\mathbf{g}_{k+1}^T \mathbf{d}_{k+1} = -\left\|\mathbf{g}_{k+1}\right\|^2 + \beta_k \mathbf{g}_{k+1}^T \mathbf{d}_k.$$

If the second term is too large, the resulting direction may fail to give adequate descent. We then restart by setting $\beta_k = 0$ and $\mathbf{d}_{k+1} = -\mathbf{g}_{k+1}$. This discards the accumulated momentum. PR+ alone does not guarantee a descent direction under an inexact line search, so a descent check is still useful.

Algorithm 3: Nonlinear conjugate gradient with PR+ and restarting

(i) Choose $\mathbf{x}_1$, tolerance $\epsilon > 0$, and a maximum number of iterations $K_{\max}$. Set $\mathbf{g}_1 = \nabla f(\mathbf{x}_1)$.

(ii) If $\|\mathbf{g}_1\| \le \epsilon$, return $\mathbf{x}_1$. Otherwise, set $\mathbf{d}_1 = -\mathbf{g}_1$ and $k = 1$.

(iii) Repeat while $k \le K_{\max}$:

- Find $\alpha_k > 0$ satisfying the strong Wolfe conditions along $\mathbf{d}_k$. If the line search fails, restart with $\mathbf{d}_k = -\mathbf{g}_k$ and retry; if it fails again, stop and report the failure.
- Set $\mathbf{x}_{k+1} = \mathbf{x}_k + \alpha_k \mathbf{d}_k$ and compute $\mathbf{g}_{k+1} = \nabla f(\mathbf{x}_{k+1})$.
- If $\|\mathbf{g}_{k+1}\| \le \epsilon$, return $\mathbf{x}_{k+1}$.
- Compute

$$\beta_k = \max\left\{0, \frac{\mathbf{g}_{k+1}^T(\mathbf{g}_{k+1} - \mathbf{g}_k)}{\mathbf{g}_k^T \mathbf{g}_k}\right\}, \qquad \mathbf{d}_{k+1} = -\mathbf{g}_{k+1} + \beta_k \mathbf{d}_k.$$

- If $\mathbf{g}_{k+1}^T \mathbf{d}_{k+1} > -\delta \left\|\mathbf{g}_{k+1}\right\|^2$, set $\beta_k = 0$ and $\mathbf{d}_{k+1} = -\mathbf{g}_{k+1}$, where, for example, $\delta = 10^{-3}$.
- Set $k = k + 1$ and repeat.

If the iteration limit is reached, return the latest iterate and report that the gradient tolerance was not met.

The method uses function values, gradients, and a few vectors of length n, without a Hessian matrix. It need not terminate within n iterations. For a nonconvex objective, a small gradient does not certify a global minimum.

Further reading: Hager and Zhang (2006), *A survey of nonlinear conjugate gradient methods*, Pacific Journal of Optimization, 2, 35–58; Nocedal and Wright (2006), *Numerical Optimization*, Chapters 3 and 5.